

An Introduction to Mercurial

Software developers need version controllers and Mercurial is a good option. It is a free, distributed source control management tool. It efficiently handles projects of any size, and offers an easy and intuitive interface. Mercurial uses the driver program, hg.

Mercurial is a version controlled system just like *git*, which keeps track of every old version of each file. It can merge different versions of your code, so that team mates can work independently on the code and then merge their changes.

Now, let's take a close look at Mercurial.

Installation

In Ubuntu, Mercurial can be installed with the following command:

```
apt-get install mercurial
```

Mercurial commands

Now that Mercurial is installed, to find out the version of Mercurial installed in the system, use the *hg version* command.

If you are stuck with a command, you can just run *hg help* which will give a list of commands, along with information on what each command does. You can use the flag *-v* along with the command to give you detailed information about the list of commands.

-v is short for *-verbose*, and it tells Mercurial to print more information than it usually would. Typing *hg* alone will also give a list of *hg* commands.

Everything in Mercurial works inside a repository. A repository is a directory where files are stored.

hg init creates a repository.

To create a copy of an existing repository in a new directory, you can run the following command:

```
hg clone option source [dest]
```

If the destination is not mentioned, then it defaults to the base name of the source.

Every Mercurial repository is independent. It has its own private copy of a project's files and history. So a cloned repository remembers the location of the repository from where it was cloned, but Mercurial will not communicate with that repository, unless you instruct it to do so. Every repository that is created consists of a directory called *.hg*, where Mercurial keeps the metadata of the repository. *hg add* adds the files to the repository,

and *hg commit* saves the current state of all files to the repository. Mercurial will pop up an editor so that you can type a commit message. This is just something you type to remind yourself of what changed in this commit.

hg log shows the history of changes committed to the repository. In the *hg log*, the *changeset* line shows two numbers to every commit: a handy short one like '0' for your initial revision, and a long hexadecimal one.

hg status shows a list of files that have changed.

hg diff shows what changes were made since the last commit.

hg remove is used to remove the file from the repository.

But the file won't be removed until the commit has been made. The same applies for *hg add* too.

hg cat is used to print any version of the file.

hg qseries is used to see the list of patches.

hg refresh is used to refresh the patch. It replaces the changes to the existing patch file.

filename.rej are rejected files, which are created when the changes are made to a file and when the file is not saved properly, i.e., when the changes are not properly applied to the tree. One way to handle them is to simply delete the *.rej* files and then apply the patch to the local tree using *hg qpush patch_name*. Another way is to simply make changes in the local tree again and run *hg*

refresh. It updates and refreshes the current patch with the new changes made to the local tree.

Knowing how to handle multiple files is another essential skill. Let's assume you are working on two patches, A and B, that have no relation with each other. This means we need to create two separate repositories for both and these should not get mixed. What we need to do is - work on Patch A, make the patch and then *qpop* it. So, here, the changes made are saved independently on Patch A, and now it is no more a part of the local tree. Next, work on Patch B and *qpop* it when you want to get back to Patch A. When we go to *hg qseries*, we get to see the list of patches. Whichever patch you want to apply, move it on top of the stack using *hg qpush -move patch_name* and then apply that patch.

I hope that helps! Thank you! 



By: H. Chaithanya

The author is an engineering student and is very passionate about open source. She has made many contributions to the Mozilla community. She can be reached at chaithanya7991@gmail.com